

**Responsable : Pr. S. Chabaa**

```

4 import seaborn as sns
5
6 from sklearn.model_selection import train_test_split
7 from sklearn.preprocessing import StandardScaler
8 from sklearn.metrics import mean_squared_error, accuracy_score
9 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
10
11 from tensorflow.keras.models import Sequential
12 from tensorflow.keras.layers import Dense
13 from tensorflow.keras.optimizers import Adam

```

Le jeu de données est chargé à partir d'un fichier CSV contenant les variables météorologiques, la puissance produite et l'indicateur de panne :

```

1 df = pd.read_csv("synthetic_solar.csv")

```

## 4 Modèle ANN pour la régression : Prediction de la puissance

### 4.1 Définition des entrées et de la sortie

Pour la tâche de régression, les variables d'entrée sont : l'irradiance solaire, la vitesse du vent et la température ambiante. La sortie du modèle correspond à la puissance produite.

```

1 X_reg = df[["Irradiance", "WindSpeed", "AmbientTemp"]].values
2 y_reg = df["Power"].values

```

### 4.2 Corrélation entre les variables

Dans cette section, nous analysons la relation entre les variables d'entrée et la variable de sortie à travers une étude de corrélation. Cette analyse vise à quantifier le degré de dépendance linéaire entre les paramètres météorologiques (irradiance solaire, vitesse du vent et température ambiante) et la puissance électrique produite par le panneau photovoltaïque. Elle permet également d'identifier les variables les plus influentes et de mieux comprendre la structure des données avant la phase de modélisation par intelligence artificielle.

```

1 # Matrice de corrélation
2 cols_corr = ['Irradiance', 'WindSpeed', 'AmbientTemp', 'Power']
3 corr = df[cols_corr].corr()
4 plt.figure(figsize=(8,6))
5 sns.heatmap(corr, annot=True, cmap="viridis", fmt=".2f", cbar=True,
6             square=True)
7 plt.title("Correlation_Matrix:_Time_Features_vs_Output", fontsize=14)
8 plt.tight_layout()

```

```
8 plt.show()
```

### 4.3 Séparation et normalisation des données

Les données sont séparées en un ensemble d'apprentissage (80 %) et un ensemble de test (20 %). Une normalisation est appliquée afin d'améliorer la convergence de l'apprentissage.

```
1 Xr_train, Xr_test, yr_train, yr_test = train_test_split(  
2     X_reg, y_reg, test_size=0.2, random_state=42)  
3  
4 scaler_reg = StandardScaler()  
5 Xr_train = scaler_reg.fit_transform(Xr_train)  
6 Xr_test = scaler_reg.transform(Xr_test)
```

### 4.4 Architecture et entraînement du réseau

Le réseau de neurones utilisé pour la régression est composé de deux couches cachées de 32 neurones avec une fonction d'activation ReLU, suivies d'une couche de sortie linéaire.

```
1 reg_model = Sequential([  
2     Dense(32, activation="relu", input_shape=(3,)),  
3     Dense(32, activation="relu"),  
4     Dense(1)  
5 ])
```

Le modèle est entraîné en minimisant l'erreur quadratique moyenne (MSE) à l'aide de l'optimiseur Adam.

```
1 reg_model.compile(optimizer=Adam(0.001), loss="mse")  
2 # --- Entraînement  
3 reg_model.fit(  
4     Xr_train, yr_train,  
5     validation_split=0.2,  
6     epochs=100,  
7     batch_size=32,  
8     verbose=0)
```

### 4.5 Test et Évaluation des performances

La performance du modèle est évaluée à l'aide de la racine de l'erreur quadratique moyenne (RMSE), définie par :

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

Une comparaison graphique entre la puissance mesurée et la puissance prédite permet d'apprécier la qualité de l'estimation.

```
1 # --- Pr diction
2 yr_pred = reg_model.predict(Xr_test).flatten()
3 # --- valuation (RMSE)
4 rmse = np.sqrt(mean_squared_error(yr_test, yr_pred))
5 print(f"RMSE:{rmse:.2f}W")
6 # --- Visualisation
7 plt.figure(figsize=(12, 5))
8 plt.plot(yr_test, label="Puissance_mesuree(W)")
9 plt.plot(yr_pred,"k--", label="Puissance_predite(W)")
10 plt.xlabel("Indice")
11 plt.ylabel("Puissance(W)")
12 plt.title("")
13 plt.legend()
14 plt.grid(True)
15 plt.show()
16 #--- Zoom In
17 plt.figure(figsize=(12, 5))
18 plt.plot(yr_test[:100], label="Puissance_mesur e(W)")
19 plt.plot(yr_pred[:100],"k--", label="Puissance_predite(W)")
20 plt.xlabel("Indice")
21 plt.ylabel("Puissance(W)")
22 plt.title("")
23 plt.legend()
24 plt.grid(True)
25 plt.show()
```

## 5 Modèle ANN pour la classification des pannes

### 5.1 Définition des entrées et de la sortie

Pour la classification, la puissance mesurée est ajoutée aux variables d'entrée. La sortie du modèle est une variable binaire indiquant l'état du système :

- 0 : fonctionnement normal,
- 1 : présence d'une panne.

```
1 X_clf = df[["Irradiance", "WindSpeed", "AmbientTemp", "Power"]].
    values
2 y_clf = df["Failure"].values
```

## 5.2 Architecture du réseau de classification

Le modèle de classification possède une structure similaire à celle du modèle de régression, avec une couche de sortie sigmoid adaptée à une classification binaire.

```
1 clf_model = Sequential([
2     Dense(32, activation="relu", input_shape=(4,)),
3     Dense(32, activation="relu"),
4     Dense(1, activation="sigmoid")
5 ])
```

## 5.3 entraînement et Test

```
1 clf_model.compile(
2     optimizer=Adam(0.001),
3     loss="binary_crossentropy",
4     metrics=["accuracy"])
5 # --- Entraînement
6 clf_model.fit(
7     Xc_train, yc_train,
8     validation_split=0.2,
9     epochs=100,
10    batch_size=32)
11 # --- Test
12 yc_pred = (clf_model.predict(Xc_test) > 0.5).astype(int).flatten()
```

## 5.4 Évaluation de la classification

La performance du classifieur est évaluée à l'aide :

- du taux de bonne classification (accuracy),
- de la matrice de confusion, permettant d'analyser les erreurs de décision.

La matrice de confusion offre une vision détaillée des faux positifs, faux négatifs, vrais positifs et vrais négatifs.

```
1 # --- Evaluation (Accuracy)
2 accuracy = accuracy_score(yc_test, yc_pred)
3 print(f"Accuracy:{accuracy*100:.2f}%")
4 # --- Matrice de confusion
5 cm = confusion_matrix(yc_test, yc_pred)
6 disp = ConfusionMatrixDisplay(cm, display_labels=["Normal", "Panne"])
7 disp.plot(cmap="Blues")
8 plt.title("Matrice_de_confusion")
9 plt.show()
```

```

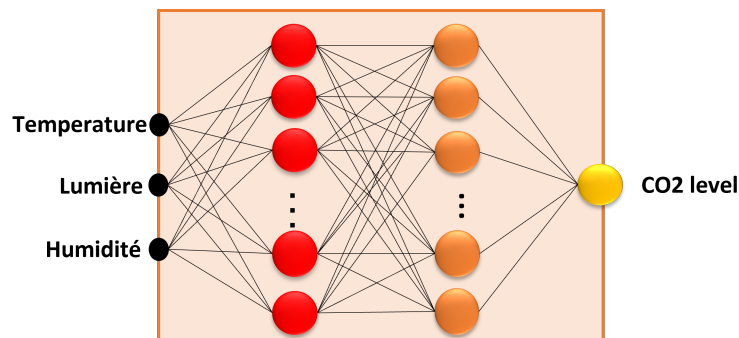
10 # VISUALISATION DES DEUX CLASSES (NORMAL / PANNE)
11 plt.figure(figsize=(7, 5))
12 # Classe normale
13 plt.scatter(
14     df[df["Failure"] == 0]["Irradiance"],
15     df[df["Failure"] == 0]["Power"],
16     alpha=0.5,
17     label="Normal"
18 )
19 # Classe panne
20 plt.scatter(
21     df[df["Failure"] == 1]["Irradiance"],
22     df[df["Failure"] == 1]["Power"],
23     alpha=0.5,
24     label="Panne"
25 )
26 plt.xlabel("Irradiance_(W/m )")
27 plt.ylabel("Puissance_(W)")
28 plt.title("Visualisation_des_classes:_Normal_vs_Panne")
29 plt.legend()
30 plt.grid(True)
31 plt.show()

```

## Exercice 2 : Estimation du taux de CO2 (Régression)

L'objectif de cette partie est de prédire une grandeur continue, la concentration de **CO2** (en ppm), en fonction des autres paramètres environnementaux (Température, Humidité, Lumière, Occupation). C'est dans cet exercice que nous étudierons en détail l'influence des hyperparamètres du réseau de neurones.

- **Entrées (X)** : Temperature, Humidity, Light.
- **Sortie (Y)** : CO2.



## A. Importance des variables

Avant de lancer l'apprentissage, il est crucial d'analyser l'impact des variables d'entrée sur la sortie. D'après la matrice de corrélation, quelle variable semble la plus corrélée avec le CO2 ?

## B. Étude des Hyperparamètres

Dans cette partie, vous allez faire varier les paramètres du réseau afin de minimiser l'erreur.

### Protocole expérimental :

- **Nombre d'épochs** : 30
- **Fonction de perte** : mean\_squared\_error (MSE)
- **Optimiseur** : adam
- **Métrique** : Relevez la **RMSE** sur le jeu de **Test**.

### 1. Influence du nombre de couches cachées

Configuration fixée : 16 neurones par couche, Activation ReLU.

| Essai | Nb Couches | RMSE (Test) | Observation |
|-------|------------|-------------|-------------|
| 1     | 1          | ...         | ...         |
| 2     | 2          | ...         | ...         |
| 3     | 3          | ...         | ...         |
| 4     | 4          | ...         | ...         |
| 3     | 5          | ...         | ...         |

### 2. Influence du nombre de neurones par couche

Configuration fixée : utiliser le nombre optimal de couches cachées trouvé dans la question 1, avec l'activation ReLU.

| Essai | Neurones/couche | RMSE (Test) | Temps de calcul (Faible/Élevé) |
|-------|-----------------|-------------|--------------------------------|
| 1     | 16              | ...         | ...                            |
| 2     | 32              | ...         | ...                            |
| 3     | 64              | ...         | ...                            |
| 4     | 128             | ...         | ...                            |

### 3. Influence de la fonction d'activation

Configuration fixée : utiliser le nombre optimal de couches cachées et le nombre de neurones optimal par couche trouvés dans la question 1 et 2, respectivement selon le tableau suivant.

| Essai | Activation | RMSE (Test) | Convergence (Rapide/Lente) |
|-------|------------|-------------|----------------------------|
| 1     | ReLU       | ...         | ...                        |
| 2     | Sigmoid    | ...         | ...                        |
| 3     | Tanh       | ...         | ...                        |

La figure 1 représente les fonctions d'activation utilisées dans cette partie.

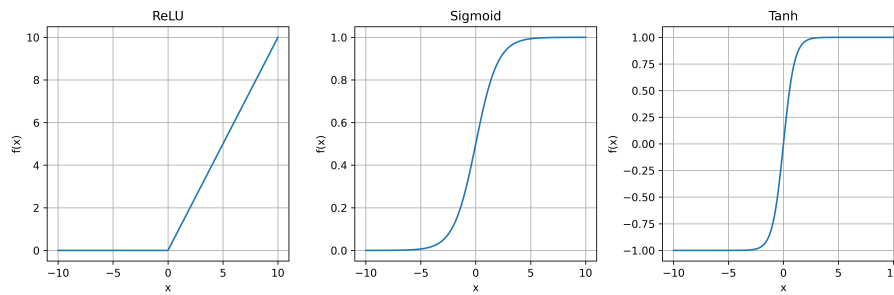


FIGURE 1 – Fonctions d'activation.

### C. Modèle Optimal et Visualisation

1. Sélectionnez la meilleure configuration (nombre de couches, neurones, activation) d'après vos résultats précédents.
2. Ré-entraînez ce modèle final sur **50 epochs**.
3. **Visualisation** : Tracez sur le même graphique :
  - La courbe du **CO2 réel** (axe des ordonnées) en fonction du temps/index.
  - La courbe du **CO2 prédit** par votre modèle.

*Astuce : Pour une meilleure lisibilité du graphique, affichez uniquement les 100 premiers points du jeu de test.*